PES UNIVERSITY
Department of Computer Science and Engineering

Software Requirements Specification

for

UATC - Universal Adaptive Transform Compression

A block-adaptive, lossless file compression and archival tool with a web-based frontend

Version 1.0
Prepared: September 2026

Team Member	SRN	PESU ID
S Chakshu	PES1UG24CS390	PES1202403582
Sai Rishi Gangarapu (Team Lead)	PES1UG24CS400	PES1202400086
Sampada K	PES1UG24CS415	PES1202402248
Manasa K	PES1UG25CS824	PES1202503000

Project Guide: Dr. Sapna V M
Course: Level 3 Software Engineering Mini Project

Revision History

Date	Version	Description	Author
Sep 2026	1.0	Initial SRS drafted from approved project synopsis	Team UATC

1. Introduction

1.1 Purpose

This document is the Software Requirements Specification (SRS) for UATC (Universal Adaptive Transform Compression). It is written based on the project synopsis approved for this semester and is meant to give a detailed, testable description of what the system is supposed to do before we start implementation. The intended readers are our project guide, the evaluation panel, and ourselves as the development team, since this document is what the design and test cases in later phases will be checked against.

1.2 Scope

UATC is a lossless, block-adaptive file compression and archival tool. It takes arbitrary files or directories as input, splits them into blocks, and for every block chooses a reversible transform that is best suited to that block instead of using one fixed pipeline for all data (which is what tools like gzip and Zstandard effectively do). Blocks that will not benefit from compression are stored as-is instead of being forced through the pipeline. The system exposes two interfaces to the user - a command line interface (CLI) for technical/scriptable use, and a browser-based web frontend for users who do not want to deal with command line arguments. Compression and decompression can use multiple CPU threads since blocks are independently decodable.

The product does not attempt to beat established compressors like Zstandard on ratio or speed - the actual goal is a correct, well-tested, adaptive compression pipeline that is benchmarked honestly against these tools. Lossy compression, audio/video codecs, a native desktop app, and distributed/multi-node execution are explicitly out of scope for this semester (see Section 2.5 and the synopsis, Section 6).

1.3 Definitions, Acronyms and Abbreviations

Term	Meaning
UATC	Universal Adaptive Transform Compression - name of the system and the archive file extension (.uatc)
Block	A bounded-size chunk of an input file that is analyzed and transformed independently
Transform	A reversible operation applied to a block before entropy coding (e.g. delta, XOR, RLE, byte-shuffle)
Entropy coder	The final lossless coding stage applied to a block's chosen representation
Compressibility probe	A lightweight check used to estimate whether a block is worth compressing at all
Store (as a decision)	Keeping a block's original bytes unchanged because no transform reduces its size enough
CLI	Command Line Interface
SRS	Software Requirements Specification (this document)

Term	Meaning
Thread pool	A fixed set of worker threads used to process independent blocks in parallel

1.4 References

- UATC Project Synopsis - Web Frontend (Final), PES University, 2026
- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications (used as a general template/checklist)
- gzip / DEFLATE (RFC 1951) documentation - used as a comparison baseline
- Zstandard (RFC 8878) documentation - used as a comparison baseline

1.5 Overview

Section 2 gives the overall picture of the product - what it does, who uses it, and what environment it runs in. Section 3 lists the actual requirements in detail: functional requirements (numbered FR-xx, carried over from the synopsis's F01-F15 list but written as shall-statements), interface requirements, and non-functional requirements. Section 4 walks through a few representative use cases/usage scenarios. Section 5 covers the archive format since that is a requirement in itself. The appendix has the team ownership split and the phase-wise plan of work.

2. Overall Description

2.1 Product Perspective

UATC is a new, standalone system - it is not an add-on to an existing product. It is organised as a layered architecture (frontend/API/compression core), so that the C++ compression core can be built and tested on its own, independent of the JavaScript web frontend. The high-level flow is:

Web Frontend (HTML/CSS/JS) or CLI -> API / Application layer -> Compression Manager -> File Analyzer -> Compressibility Probe -> Block Manager / Archive Index -> Adaptive Transform Engine (None / Delta / XOR / Byte-shuffle / RLE) -> Entropy Coding Engine -> Thread-pool Scheduler -> Archive Builder -> .uatc archive

The web frontend only handles file selection, triggering operations, and showing progress/results/errors. All actual compression logic - transforms, entropy coding, archive handling, integrity checks, and parallel scheduling - lives in the C++ backend and is reached through an HTTP application interface.

2.2 Product Functions (Summary)

- Compress a single file or an entire directory tree into a .uatc archive
- Extract a .uatc archive back to the original files/folder structure, byte-for-byte
- Analyse each block and estimate how compressible it is before doing real work
- Pick the best reversible transform per block based on estimated cost, or store the block unchanged
- Run entropy coding on the chosen representation

- Process independent blocks in parallel using a thread pool
- Verify archive/block integrity using checksums, and report corruption instead of failing silently
- Show archive statistics - size, ratio, block/transform distribution, timing
- Provide the above through both a CLI and a web-based UI

2.3 User Classes and Characteristics

User Class	Description	Technical Level
Non-technical end user	Wants to compress/extract files or folders through a browser, without knowing anything about compression algorithms or the CLI	Low
Technical / power user	Uses the CLI to script compression jobs, automate backups, or integrate UATC into other tools	High
Developer / student (us)	Needs a scriptable, reproducible tool with visible statistics and integrity checks for testing and evaluation	High
Project evaluators	Need to see clear requirements, measurable performance numbers, and an explainable, testable architecture	High

2.4 Operating Environment

- Backend/core: C++17, built with CMake, targeting standard desktop OS environments (no OS-specific dependencies beyond what's unavoidable)
- CLI: native executable built from the same core library used by the backend
- Frontend: any modern browser supporting HTML5/CSS3/JS, communicating with the backend over HTTP
- Development/testing machines: standard multi-core lab/personal machines used by the team for benchmarking

2.5 Design and Implementation Constraints

- Core must be written in C++17 - chosen for low-level byte control, performance, and concurrency support
- The web frontend must not implement any compression logic itself - it is purely presentation/control
- Archive format must be versioned from the start so future transforms don't break older archives
- The candidate transform set for v1 is fixed to a small list (none, delta, XOR, RLE, byte-shuffle, and bit-plane organisation is being considered) - one extra experimental transform is a stretch goal only, and only after the MVP is stable
- No guarantee that every input gets smaller - this is a functional requirement violation only if the store-vs-compress decision itself is wrong, not if a block simply doesn't shrink
- Semester timeline (6 phases, see Appendix B) constrains how much can be attempted - correctness of the core pipeline is prioritised over feature count

2.6 Assumptions and Dependencies

- We're assuming block-level parallelism is enough to get a real throughput benefit on multi-core dev/test machines - we haven't measured this yet, it's a benchmarking goal for Phase 4
- We're assuming the test files/datasets we use for benchmarking (text, source code, numeric data, structured binary, already-compressed data) are representative enough to justify the transform choices
- Depends on a standard entropy coding library/approach being available and usable from C++ without heavy licensing issues
- Assumes team members have working C++/CMake toolchains and Git set up individually - this is not something the SRS can guarantee but it's a precondition for the plan of work

3. Specific Requirements

3.1 Functional Requirements

The functional requirements below are written as testable shall-statements. The ID in brackets refers back to the corresponding feature ID (F01-F15) in the approved synopsis, so the two documents stay traceable to each other.

ID	Requirement
FR-01 (F01)	The system shall accept a single file or a directory as input and enumerate all files/sub-folders required for archival.
FR-02 (F02)	The system shall create a .uatc archive containing global metadata, a file table, a block index, and the compressed/stored block payloads.
FR-03 (F03)	The system shall extract a .uatc archive and reconstruct the original files and directory structure with byte-for-byte content fidelity.
FR-04 (F04)	The system shall compute lightweight per-block statistics that are used as input to the compressibility and transform-selection decisions.
FR-05 (F05)	The system shall estimate, before doing full compression work, whether a block is likely to shrink enough to justify that effort (compressibility probe).
FR-06 (F06)	The system shall evaluate the defined set of candidate reversible transforms for a compressible block and select the transform with the lowest estimated encoded cost.
FR-07 (F07)	The system shall apply a lossless entropy coding stage to the block representation chosen by the transform-selection step.
FR-08 (F08)	The system shall store a block's original bytes unchanged whenever the compressed form (including metadata overhead) would not be smaller than the original.
FR-09 (F09)	The system shall validate block-level and archive-level checksums during extraction/verification and shall report corruption clearly instead of producing silently-wrong output.
FR-10 (F10)	The system shall process independent blocks concurrently using a thread pool, and the resulting archive shall be identical regardless of whether processing happened serially or in parallel.
FR-11 (F11)	The system shall be able to display archive information on request - total size, file count, block count, transform usage breakdown, and compression statistics - without doing a full extraction.
FR-12 (F12)	The system shall detect malformed archives, missing blocks, checksum failures, and unsupported archive-format versions, and report each of these as a distinct, understandable error.
FR-13 (F13)	The system shall provide a browser-based interface through which a non-technical user can select files/folders and start compression or extraction operations.
FR-14 (F14)	The web interface shall display operation progress, completion status, errors, and the resulting compression statistics in a way a non-technical user can understand.

ID	Requirement
FR-15 (F15)	The JavaScript frontend and the C++ backend shall exchange operation requests, progress updates, results, and error information through a defined application interface (HTTP API).

3.2 External Interface Requirements

3.2.1 User Interfaces

- CLI: a native command-line executable exposing at least compress, extract, info, verify, and benchmark sub-commands (see Section 4 for exact usage)
- Web UI: a browser page that lets the user pick files/folders, choose Compress or Extract, choose a destination, start the operation, and see live progress plus a results/statistics screen; errors must be shown in plain language, not raw backend error codes

3.2.2 Hardware Interfaces

No special hardware is required. The system should run on any standard multi-core desktop/laptop used for development and evaluation. Multiple CPU cores are used where available but the system must still function correctly (just slower) on a single core.

3.2.3 Software Interfaces

- Operating system: any OS the C++17 toolchain and CMake support - the core avoids OS-specific dependencies where possible
- Browser: any current browser with standard HTML5/CSS3/JS support, for the web frontend
- Build tooling: CMake for building the C++ core/CLI; Git/GitHub for version control and collaboration

3.2.4 Communication Interfaces

The web frontend talks to the C++ backend over an HTTP-based application interface. This interface must be able to carry operation requests (e.g. start compression of a given path), progress/status updates while an operation is running, and final results or error information.

3.3 Non-Functional Requirements

	Category	Requirement
NFR-01	Correctness	Every file successfully extracted from a UATC archive must match the original input byte-for-byte. This is treated as a hard requirement, not a best-effort goal.
NFR-02	Performance	Compression/decompression throughput should measurably improve with additional worker threads for sufficiently large inputs, subject to I/O limits. Performance is to be reported honestly against gzip/DEFLATE and Zstandard, not framed as beating them.
NFR-03	Reliability	Corrupted archives and checksum mismatches must always be detected and reported. The system must never silently produce incorrect output.
NFR-04	Usability	Core operations must be reachable through simple CLI commands and through the web UI with visible progress and statistics, without the user needing to understand the underlying algorithms.

	Category	Requirement
NFR-05	Maintainability	Transforms, the entropy coder, archive handling, and the scheduler must be modular components so a new transform can be added without redesigning the archive format, API, frontend, or CLI.
NFR-06	Portability	The compression core targets standard C++17 and should avoid unnecessary OS-specific dependencies.
NFR-07	Observability	The system must record and expose compression ratio, elapsed time, throughput, and which transform was selected per block.
NFR-08	Responsiveness	The web interface must remain usable (not freeze) while a long-running compression/extraction job executes in the backend.

3.4 Data Requirements - Archive Format

A .uatc archive is required to contain the following components (this is treated as a requirement, not just a design note, because correctness and versioning depend on it):

Component	Purpose
Magic + Version	Identifies the file as a UATC archive and the format/decoder version it needs
Global Metadata	Archive-level flags, creation info, block-size policy, configuration used
File Table	Original relative paths, file sizes, and timestamps/attributes where applicable
Block Index	Per-block offsets, original length, stored length, transform ID, codec ID, checksum
Block Payloads	The actual compressed or raw block data, each independently decodable
Integrity Data	Archive-level and/or file-level checksums used during verification

4. Use Cases / Usage Scenarios

4.1 UC-1: Non-technical user compresses a folder (Web UI)

Actor: Non-technical end user.

Precondition: User has the UATC web page open in a browser and the backend is reachable.

Main flow: 1) User opens UATC in the browser. 2) User selects a folder to compress. 3) User chooses "Compress" and picks a destination. 4) User starts the operation. 5) The UI shows live progress. 6) On completion the UI shows the resulting archive size, ratio, and time taken.

Alternate flow: if the operation fails partway (e.g. disk full, permission error), the UI must show a clear, non-technical error message instead of a stack trace or raw error code.

4.2 UC-2: Technical user compresses a dataset with 8 worker threads (CLI)

Actor: Technical/power user.

Command: uatc compress ./dataset.bin ./dataset.uatc --workers 8

Expected result: dataset.bin is split into blocks, each block is analysed and transformed adaptively, blocks are processed across 8 threads, and dataset.uatc is produced. Running the same compression with --workers 1 must produce a byte-identical archive (per FR-10).

4.3 UC-3: Verifying and inspecting an archive

Commands: uatc verify ./project.uatc (checks metadata and checksums without extracting)

uatc info ./project.uatc (shows size, file/block counts, and transform-usage statistics)

Expected result: if the archive is intact, verify reports success; if any block/checksum fails, it must report exactly what failed rather than just "error".

4.4 UC-4: Extraction

Command: uatc extract ./project.uatc ./restored

Expected result: ./restored contains the original directory structure and every file matches the original byte-for-byte (verified via checksum comparison in testing).

4.5 UC-5: Benchmarking against gzip/Zstandard

Command: uatc benchmark ./samples

Expected result: a repeatable report comparing UATC's ratio, speed, and resource usage against gzip/DEFLATE and Zstandard on the same sample set, used as evidence in Phase 6 (Validation and Demonstration).

Appendix A - Team-wise Functional Ownership

This is the same ownership split from the synopsis, included here for traceability between requirements and who is responsible for building/testing them. Split may be adjusted as work progresses.

Owner	Primary Area	Related Requirements
Sai Rishi Gangarapu (Lead)	Compression core and adaptive transform engine	FR-04, FR-05, FR-06, FR-07
S Chakshu	Archive / file-system layer	FR-01, FR-02, FR-03, FR-09
Sampada K	Execution and performance layer	FR-10, plus benchmarking/perf non-functional requirements
Manasa K	Archive inspection/verification, web frontend, integration	FR-11, FR-12, FR-13, FR-14, FR-15

Appendix B - Plan of Work

Phase	Milestone	Main Outputs
Phase 1	Project setup and requirements	Final synopsis, this SRS, repository setup, module boundaries, test strategy
Phase 2	MVP compressor	Byte-block reader/writer, baseline lossless coder, basic archive, extraction, checksums
Phase 3	Adaptive engine	Compressibility probe, transform interface, candidate transforms, cost-based selection
Phase 4	Parallelism and benchmarking	Thread pool, parallel block processing, benchmark harness, profiling
Phase 5	Hardening	Corruption tests, edge cases, regression tests, CLI polish, archive versioning
Phase 6	Validation and demonstration	Comparative benchmarks vs gzip/Zstandard, correctness evidence, final docs and demo

Stretch goal (only after Phase 4, if time permits): one extra experimental reversible transform added behind the existing transform interface, without changing the archive format.

Appendix C - Sign-off

Role	Name	Signature
Project Guide	Dr. Sapna V M	
Team Lead	Sai Rishi Gangarapu	
Team Member	S Chakshu	
Team Member	Sampada K	

Role	**Name**	**Signature**
Team Member	Manasa K	